# DFSha

## Sistema de archivos distribuido por bloques

Proyecto 1 - ST0263 / SI3007 - Universidad EAFIT - 2026-2. Integrantes: Camilo, Mendel y Martín. Fecha del informe: 26 de octubre de 2026 (America/Bogota).

Entrega técnica local. La publicación en AWS/GCP y la validación por Internet están fuera de este cierre por indicación del equipo; no se presentan como requisitos cumplidos.

### Resumen

DFSha separa el namespace y los metadatos de la transferencia de bytes. Dos ControlNodes comparten PostgreSQL; cuatro DataNodes guardan bloques cifrados; un cliente CLI consulta manifiestos y transmite directamente a los nodos. RabbitMQ coordina heartbeats, réplicas y borrados. La solución incorpora edición por versiones con escritor exclusivo, replicación eventual, recuperación de publicaciones de borrado y TLS en el perfil seguro.

La ejecución local completa del 26 de octubre aprobó integridad de 20 MiB, tres réplicas por bloque, RF3, aislamiento, locks, cuatro clientes concurrentes y fallos controlados de DataNode, ControlNode, broker y PostgreSQL. La batería Maven aprobó 89 pruebas, 49 con PostgreSQL real. Las limitaciones se explicitan: un host, un broker, promoción SQL manual y replicación asíncrona no equivalen a disponibilidad o durabilidad de producción.

### Trazabilidad

Repositorio: https://github.com/cameloxd/proyectoTelematica

Código de la demo: 74c560bccd3976be370954c4250008e06a46d917. El informe se añade después de esa ejecución, sin atribuir la evidencia a un commit futuro. Recibo: docs/entrega/evidencia/resultado.json; suite: pruebas-maven.json en la misma carpeta. Los hashes de los JAR ejecutados se conservan en el recibo. La ubicación final de la entrega debe verificarse en GitHub después de integrar develop y main.

# 1. Objetivos, problema y fundamentos

## Objetivo general

Construir y validar un servicio distribuido de archivos que oculte la ubicación física de los bloques, preserve integridad y aislamiento, y continúe leyendo ante fallos controlados de componentes cuando exista una copia o servicio alternativo disponible.

## Objetivos específicos

- Ofrecer namespace jerárquico por propietario y autenticación de usuarios/nodos.
- Particionar archivos, distribuir primarias, replicar y recuperar copias faltantes.
- Implementar send/receive y open/close/read/write/lock con semántica explícita.
- Cifrar bloques y conexiones del perfil seguro, verificando CA y nombre del servidor.
- Probar concurrencia, consistencia e integridad con resultados reproducibles y acotados.

## Problema que resuelve

Guardar todos los bytes en un único proceso mezcla administración de nombres y tráfico de archivos, concentra capacidad y convierte una caída de ese almacenamiento en pérdida de acceso. DFSha distribuye los bloques y expone un único namespace lógico. El usuario no elige nodos: el ControlNode devuelve un manifiesto y el cliente selecciona una copia.

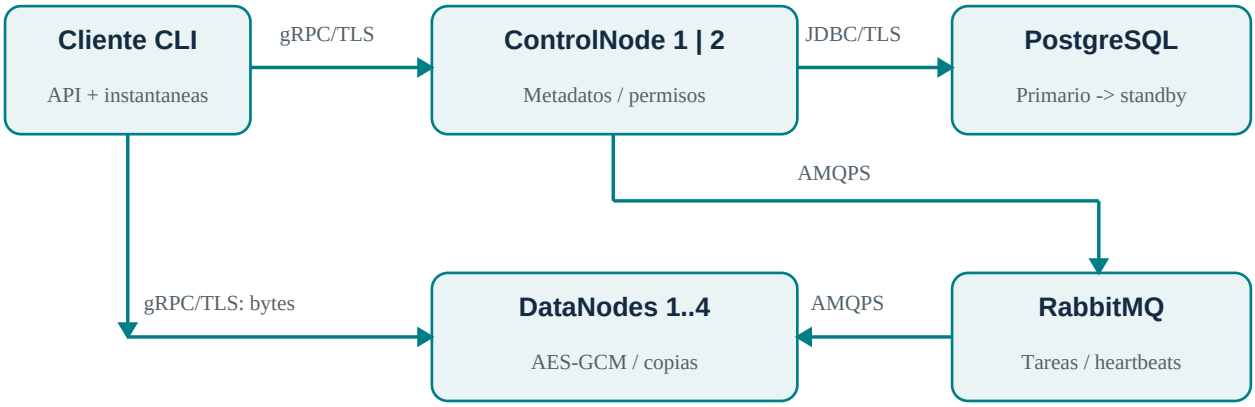
## Marco teórico breve

GFS estudia almacenamiento distribuido para aplicaciones intensivas en datos y trata la tolerancia a fallos y el rendimiento agregado como objetivos centrales [1]. Es una referencia conceptual, no un componente ni una implementación que DFSha reproduzca.

HDFS separa la gestión de namespace y ubicaciones del almacenamiento/servicio de bloques; los datos de usuario no atraviesan el NameNode [2]. DFSha adopta esa separación de planos, pero persiste metadatos en PostgreSQL y usa dos ControlNodes activos, no el esquema exacto de HDFS.

En este proyecto, consistencia significa confirmar namespace/versiones mediante transacciones y validar identidad de operación y lease. La replicación de bytes es eventual: el commit requiere una copia escrita de cada bloque, no todas las réplicas objetivo. Disponibilidad y durabilidad se evalúan por escenarios concretos, sin extrapolar a cualquier fallo.

# 2. Arquitectura y responsabilidades




Los bytes del archivo NO atraviesan ControlNode. Diagrama propio de DFSha.

## Plano de control

ControlServiceImpl implementa usuarios, directorios, asignación, commit, manifiestos y locks. NodeRegistryImpl registra DataNodes y valida sus reportes. PostgreSQL conserva usuarios, versiones, bloques, ubicaciones, nodos, leases y outbox. El estado durable es compartido: las conexiones y trabajadores de cada ControlNode sí son locales al proceso.

## Plano de datos

DfsClient divide el trabajo y abre streams directamente a los DataNodes. DataNodeServiceImpl valida tokens, tamaños y hashes; BlockStore cifra y persiste. Replicator obtiene una copia desde otro DataNode usando el mismo ReadBlock. El ControlNode no recibe los bytes completos del archivo.

## Plano asíncrono

ControlLink publica heartbeats y consume tareas. ClusterMonitor detecta estados y reserva réplicas faltantes. DeleteOutbox publica tareas guardadas junto al borrado lógico. RabbitMQ desacopla estas tareas del tiempo de respuesta al usuario. Las colas durables y confirms no eliminan la necesidad de recuperar consumidores/DLQ.

## Reparto de explicación y mantenimiento

- Camilo: metadatos, autenticación/token, reservas, colocación, outbox y planificación de copias.
- Mendel: cliente, archivos abiertos, almacenamiento, cifrado y transferencia/réplica física.
- Martín: contrato, comunicaciones, TLS/AMQP, heartbeats e infraestructura.

Son responsabilidades de revisión, no autoría exclusiva. RF3 y TLS atraviesan varios módulos; se mantienen una sola implementación y un solo contrato compartido.

# 3. Protocolos, contrato y API

## gRPC y autenticación

El proto v1.2 genera Java en co.eafit.dfsha.grpc.v1. ControlService atiende al cliente; NodeRegistryService registra/reporta nodos; DataNodeService transfiere bloques. gRPC usa TLS en la demo. Las sesiones viajan como Bearer; los tokens de bloque llevan tipo, UUID y expiración firmados con Ed25519. El DataNode verifica con la clave pública.

- RF1: Register/Login, ListDirectory, CreateDirectory y DeletePath; CLI añade cd/pwd.
- RF2: AllocateFile, WriteBlock, ReportBlockStored, CommitFile y GetFileManifest/ReadBlock.
- RF3: API DfsClient.open y RemoteFile.read/write/close/abort, más AcquireLock/ReleaseLock.
- AllocateFileRequest conserva campos 1-3 y añade replaces_file_id (4) y lock_token (5).

No renumera campos ni introduce un segundo contrato para RF3.

- WriteBlock recibe stream de cliente; ReadBlock devuelve stream de servidor.

Los trozos son de 1 MiB; no se afirma un RPC bidireccional único.

## Semántica de archivo abierto

open(r) guarda una instantánea completa en caché local privada. open(rw) adquiere un lease exclusivo de 600 s y registra el file_id esperado. read/write operan por offset en bytes; la CLI convierte texto a UTF-8. La API de escritura acepta bytes. La lectura por handle está acotada a 64 MiB por llamada.

write cambia solo la caché. close sube una versión nueva y publica su reemplazo atómicamente si versión y lease siguen vigentes. El lector abierto conserva la instantánea previa. abort, exit o cierre del cliente descartan handles pendientes, sin confirmación implícita. No hay renovación automática del lease, montaje FUSE ni semántica POSIX completa.

## Mensajería y SQL

RabbitMQ transporta protobuf de Heartbeat, ReplicationTask y BlockDeleteTask; no son RPC. El publicador de borrados usa mensajes persistentes, mandatory y confirms, y solo después actualiza published_at. Una confirmación ambigua puede producir un reenvío: la garantía es al menos una vez. Un error permanente de consumidor va a DLQ y requiere intervención.

JDBC usa consultas parametrizadas y transacciones. El perfil seguro exige verify-full con CA y hostname; el driver multi-host busca al servidor con rol primary después de la promoción.

# 4. Algoritmos y consistencia

## Partición y colocación

Para S bytes y bloque B = 8 MiB se crean ceil(S/B) bloques; un archivo vacío no crea bloques. La carga de 20 MiB se divide en 8 + 8 + 4 MiB. Solo compiten nodos ALIVE con capacidad reservable y menos de MAX_TRANSFERS_PER_NODE transferencias activas.

Para primarias se eligen primero los nodos con menos bloques de ese archivo en la ronda. Entre ellos gana el puntaje 0,60 espacio + 0,25 poca carga + 0,15 pocos bloques; cada componente se normaliza al intervalo [0,1] y los empates se resuelven por nombre. CapacityReservations serializa reservas de primarias y réplicas para no prometer dos veces el espacio.

## Confirmación y reemplazo

Una asignación crea PENDING y ubicaciones PLANNED. El DataNode persiste y reporta; solo copias WRITTEN cuentan para confirmar. CommitFile rechaza bloques faltantes. operation_id hace reintentable una misma asignación, sin duplicar su plan.

Una edición conserva la versión COMMITTED mientras sube la sustituta. En una transacción, el commit comprueba lease/versión, retira la anterior, añade sus tareas de borrado y confirma la nueva. Un fallo anterior al commit mantiene visible la versión vieja. La migración 002 permite ese staging y evita más de un reemplazo pendiente para la misma versión.

## Replicación y detección de fallos

El monitor revisa cada 5 s. Heartbeats cada 5 s; SUSPECT tras 15 s y DEAD tras 30 s sin latir. Planifica hasta el factor objetivo, excluye nodos que ya tienen copia y reserva destinos. El destino descarga, verifica SHA-256, cifra y reporta. El mismo mecanismo atiende archivos nuevos y nodos caídos. Un manifiesto devuelve copias escritas y el cliente prueba alternativas.

## Borrado y cargas abandonadas

DeletePath confirma estado DELETED y outbox juntos. El trabajador reintenta publicaciones fallidas y reparte filas con SKIP LOCKED. La ausencia física se consulta por separado. BlockStore persiste un tombstone antes de borrar: un UUID borrado no puede reaparecer por un mensaje atrasado. UUID vivo solo admite reenvío idéntico; bytes distintos se rechazan.

Cada 30 s se expiran lotes de cargas PENDING abandonadas. TTL por defecto 3600 s, mínimo 60, medido desde creación y sin heartbeat de carga. Se debe ajustar para transferencias largas. Se retienen tombstones y filas de diagnóstico; no se certifica recolección ilimitada de huérfanos.

# 5. Seguridad y ambiente de ejecución

## Controles implementados

Las contraseñas usan PBKDF2-HMAC-SHA256, sal aleatoria y 100 000 iteraciones. El namespace se filtra por propietario. Un DataNode necesita clave de registro y token de nodo; los permisos de bloque están firmados por ControlNode. No hay ACL por grupos, 2FA ni mTLS.

BlockStore cifra con AES-256-GCM, IV aleatorio de 12 bytes y UUID como dato asociado. La clave AES compartida se monta como secreto. Un reenvío idéntico no reescribe el cifrado; GCM detecta alteración. Los candados de disco son por proceso: un volumen debe tener un solo escritor.

El overlay seguro añade AMQPS con SSLContext explícito y verificación de hostname/CA, no trust-all. PostgreSQL copia la clave con permisos restrictivos y rechaza conexiones host sin TLS; JDBC/replicación usan verify-full. Estas opciones siguen las guías oficiales de RabbitMQ [3] y PostgreSQL [4]. La consola RabbitMQ es HTTP limitada a loopback, no forma parte de la afirmación de cifrado de datos entre aplicaciones.

## Ambiente validado

Windows y Docker Desktop con motor Linux 29.9.1; Compose 5.6.0. Aplicaciones Java con release 17, JAR compilados usando JDK 24 y ejecutados en contenedores eclipse-temurin:17-jre. El cliente de la prueba se lanzó desde la máquina anfitriona. PostgreSQL 16-alpine y RabbitMQ 3-management-alpine; las imágenes exactas de una futura pasada pueden variar.

El proyecto aislado usa ControlNodes 50071/50072, DataNodes 50081-50084, SQL 55434, AMQPS 5674 y consola 15673. Los secretos y certificados se generan por proyecto y nunca se suben. Los SAN incluyen localhost y nombres internos de los servicios.

## Reproducción segura

Compilar con Maven y ejecutar local-demo.ps1 -Secure -LoadClients 4 con nombre nuevo. Con -ValidateFailures el script detiene componentes y termina promoviendo el standby. No reutilizar ese proyecto con up: el primario antiguo debe permanecer detenido. DEMO-LOCAL.md detalla requisitos, comandos y conservación de evidencia.

El esquema efectivo es schema.sql, migración 001 (outbox) y 002 (versiones), en orden. Bases existentes requieren respaldo verificado y ventana sin escritores; recrear un contenedor no migra su volumen. El ControlNode comprueba el esquema, no ejecuta DDL por su cuenta.

# 6. Pruebas y análisis de resultados

## Batería automatizada

Maven package aprobó 89 pruebas sin fallos, errores ni omisiones: common 17, ControlNode 55, DataNode 11 y cliente 6. De ellas, 49 usan PostgreSQL real en esquemas aislados. Se cubren reservas/idempotencia/concurrencia, versiones/leases, rollback/outbox, cifrado e inmutabilidad. Los tests de cliente usan gRPC y servidor acotado; los de confirms AMQP usan canal simulado.

El contrato v1.2 fue generado desde PowerShell del usuario; el sandbox reutilizó esas fuentes verificando vigencia/hash. No se afirma una generación limpia dentro del sandbox Windows. La CI añadida usa Java 17, PostgreSQL 16 y clean verify; debe consultarse su ejecución remota, no confundirse su configuración con un resultado ya obtenido.

## Demostración integrada real

La ejecución terminó a las 01:16:13 -05:00 del 26 de octubre. El archivo de 20 MiB produjo tres bloques con al menos tres copias vivas cada uno. Original y descarga dieron SHA-256: D9BD7967570F3075F0ED345833B31C7436D3ACFD1F5109B232F0F5F6F809B247. Tras rm se comprobó outbox publicada y ausencia de esos UUID en los cuatro discos.

RF3 mostró write de LISTO en caché, lector previo con DFSHA antes/después del close y descarga final de la versión nueva. Se rechazaron plaintext SQL/gRPC, acceso de otro propietario y un segundo lock; después de expirar el lease se pudo adquirir otro. Los Error de esos logs son rechazos esperados y comprobados, no fallos ocultados.

## Concurrencia y fallos

Cuatro clientes concurrentes subieron y descargaron 20 MiB cada uno: 160 MiB agregados, 47,868 s de reloj, 3,343 MiB/s, cuatro checksums correctos. Incluye login y borrado; es una medición única local, no ancho de banda puro, percentiles o prueba de escalabilidad ilimitada.

Con un DataNode detenido se descargó desde otra réplica, se detectó DEAD y se repuso factor tres. Con un ControlNode detenido el cliente cambió de endpoint y conservó el checksum. Con RabbitMQ detenido el borrado lógico persistió tareas; al recuperarlo se verificó limpieza física. Por último se sincronizó WAL, se detuvo el primario SQL y se promovió manualmente el standby: login y lectura del texto actualizado funcionaron. No se probó pérdida abrupta de WAL pendiente.

Estos resultados apoyan las propiedades locales de los escenarios ejecutados. No demuestran ausencia de todos los errores posibles ni tolerancia a caídas simultáneas arbitrarias.

# 7. Conclusiones, límites y entrega

DFSha separa control/datos, distribuye bloques y recupera lecturas mediante copias y endpoints alternativos. La edición por versiones evita publicar contenido incompleto. Outbox y tombstones resuelven ventanas distintas: persistencia de la intención de borrar y rechazo de escrituras tardías. La evidencia combina pruebas acotadas con una ejecución distribuida real en contenedores locales.

## Límites que deben exponerse en la sustentación

- Un equipo anfitrión y un broker: no hay HA del host ni del servicio AMQP durante su caída.
- SQL asíncrono y promoción manual: no hay pérdida cero garantizada ni fencing automático.
- Commit antes del factor tres: ventana de sub-replicación.
- Caché completa, lease sin renovación y API no POSIX; salida sin close descarta cambios.
- DLQ manual, retención de tombstones/outbox, sin rebalanceo o decommission certificados.
- CA y clave AES de demo, no gestión de claves/certificados de producción.
- AWS/GCP e Internet excluidos por el equipo; no existe excepción confirmada del profesor.

## Qué debe adjuntarse

1. Este informe PDF (y, opcionalmente, su fuente Markdown).
2. URL del repositorio con código, pruebas, documentación y referencia final de entrega verificadas.
3. Video real de 10-15 minutos: explicación y demostración en vivo o simulada, identificada correctamente. El guion de 12 minutos es apoyo, no sustituye el archivo/enlace de video.
4. Evidencia sanitizada opcional: recibo, reportes y logs; sin secretos, claves ni volúmenes.

La asignación no especifica aquí una plataforma de carga, nombre único de archivo o ZIP obligatorio. Confirmar esas instrucciones directamente con el curso. No incluir credenciales para que el evaluador pueda arrancar: el generador crea secretos nuevos por despliegue.

## Referencias

[1] Ghemawat, Gobioff y Leung. The Google File System. SOSP, 2003. https://research.google/pubs/the-google-file-system/

[2] Apache Hadoop. HDFS Architecture (documentación oficial). https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[3] RabbitMQ. TLS Support (documentación oficial). https://www.rabbitmq.com/docs/ssl

[4] PostgreSQL 16. Secure TCP/IP Connections with SSL. https://www.postgresql.org/docs/16/ssl-tcp.html

Requisito fuente: asignación Proyecto 1 DFS, ST0263/SI3007, EAFIT 2026-2, entregada por Camilo. Las afirmaciones de implementación/resultados proceden del código y la evidencia de este repositorio, no de las referencias conceptuales.